

DEV_IMPL_GUIDE - EM-CFG-03 RBAC Catalog

Use with DD_EM-CFG-03-RBAC_Catalog-v1.0.md.

This guide explains how RBAC executes at runtime. It deliberately separates authentication from authorization:

- FOUNDATION_AUTH validates who the user is.
- EM-CFG-03 tells modules what the user is allowed to do, and where.

1. Runtime Components

Component	Responsibility
Keycloak	Authenticates user and issues JWT.
API gateway/module middleware	Validates JWT using FOUNDATION_AUTH.
RBAC API	Owns role, permission, assignment, scope, and effective-access APIs.
RBAC DB schema	Stores RBAC catalog and assignments.
RBAC service	Computes effective permissions and scopes.
Reference clients	Validate franchise, contractor, team, and tech-region scope values.
Redis/cache	Optional short-lived effective-access cache.
Kafka outbox dispatcher	Publishes RBAC change events for cache invalidation/audit.

For a medium deployment, RBAC can run in the EM/business-services pod. Keep package/module boundaries clear in code.

2. Before Requests Can Work

Create these tables:

```
rbac_role
rbac_permission
rbac_role_permission
rbac_user_role_assignment
rbac_user_scope_assignment
rbac_frontend_action
rbac_role_frontend_action
rbac_change_audit
idempotency_key
outbox_event
```

Seed minimum permissions before enabling frontends:

```
customer.read
ticket.read
ticket.create
subscription.read
billing.invoice.read
workorder.read
catalog.manage
franchise.manage
rbac.manage
report.view
```

Seed minimum roles:

```
SUPER_ADMIN
READ_ONLY_AUDITOR
CUSTOMER_CARE_AGENT
CUSTOMER_CARE_SUPERVISOR
KYC_REVIEWER_L1
KYC_REVIEWER_L2
BILLING_OPERATOR
BILLING_LEAD
DISPATCHER
WO_SUPERVISOR
CATALOG_ADMIN
SALES_AGENT
SALES_SUPERVISOR
FRANCHISE_MANAGER
REPORT_VIEWER
```

3. Flow 1 - Seed Role And Permission Catalog

3.1 Migration Or Admin Script Sends Role Create

```
POST /api/rbac/roles
Authorization: Bearer <jwt>
Idempotency-Key: seed-role-customer-care-agent
Content-Type: application/json

{
  "operatorCode": "WIK",
  "roleCode": "CUSTOMER_CARE_AGENT",
  "displayName": "Customer Care Agent",
  "roleFamily": "CUSTOMER_CARE",
  "description": "Can search customers, view Customer 360, and create tickets.",
  "keycloakRoleName": "CUSTOMER_CARE_AGENT"
}
```

Execution:

1. RBAC API validates JWT and requires `rbac.manage`.
2. Check idempotency key before insert.
3. Validate role code format.
4. Check (`operator_code`, `role_code`) uniqueness.
5. Insert `rbac_role` with `status = ACTIVE`.
6. Insert audit row `ROLE_CREATED`.
7. Insert outbox event `RbacRoleCreated`.
8. Store idempotency response.
9. Commit.

3.2 Add Permission To Role

```
POST /api/rbac/roles/role_01J_CC_AGENT/permissions
Authorization: Bearer <jwt>
Idempotency-Key: add-ticket-create-to-cc-agent
Content-Type: application/json

{
  "permissionCode": "ticket.create",
  "effect": "ALLOW"
}
```

Execution:

1. Validate caller has `rbac.manage`.
2. Resolve role.
3. Resolve permission.
4. Reject if role or permission is not active.
5. Upsert or insert `rbac_role_permission`.
6. Insert audit row `ROLE_PERMISSION_ADDED`.
7. Insert outbox event `RbacRolePermissionChanged`.
8. Commit.

4. Flow 2 - Assign Role To User

4.1 Backoffice Sends Request

```
POST /api/rbac/users/kc_user_alice/roles
Authorization: Bearer <jwt>
Idempotency-Key: assign-alice-cc-agent
```

Content-Type: application/json

```
{
  "operatorCode": "WIK",
  "roleCode": "CUSTOMER_CARE_AGENT",
  "source": "BACKOFFICE_ADMIN",
  "effectiveFrom": "2026-05-29T11:00:00Z",
  "reasonCode": "NEW_HIRE"
}
```

4.2 RBAC Service Executes

1. Authenticate admin JWT.
2. Check admin has `rbac.manage`.
3. Check idempotency key and request hash.
4. Validate target user exists in Keycloak or user registry.
5. Resolve role by (`operatorCode`, `roleCode`).
6. Reject if role is not active.
7. Check no duplicate active assignment exists for same user/role/operator.
8. Insert `rbac_user_role_assignment`.
9. Insert audit row `USER_ROLE_ASSIGNED`.
10. Insert outbox event `RbacUserRoleAssigned`.
11. Invalidate effective-access cache for `kc_user_alice`.
12. Commit.

Response:

```
{
  "assignmentId": "ura_01J_ALICE_CC",
  "authUserId": "kc_user_alice",
  "operatorCode": "WIK",
  "roleCode": "CUSTOMER_CARE_AGENT",
  "active": true
}
```

5. Flow 3 - Assign Scope To User

5.1 Backoffice Sends Request

POST `/api/rbac/users/kc_user_alice/scopes`
Authorization: Bearer `<jwt>`
Idempotency-Key: `assign-alice-franchise-nrb002`
Content-Type: application/json

```
{
  "operatorCode": "WIK",
  "scopeType": "FRANCHISE",
  "scopeValue": "fr_01J_NRB_002",
  "scopeLabel": "Nairobi 002",
  "effectiveFrom": "2026-05-29T11:00:00Z",
  "reasonCode": "FRANCHISE_TEAM_ASSIGNMENT"
}
```

5.2 RBAC Service Validates Scope

Validation depends on `scopeType`:

Scope type	Validation client
OPERATOR	Operator config.
FRANCHISE	EM-01 GET <code>/api/franchises/{id}</code> .
TECH_REGION	ILM-CFG-02/RLM region API.
CONTRACTOR	EM-02 GET <code>/api/contractors/{id}</code> .
TEAM	EM-02 team/agent registry API.

Scope type	Validation client
CHANNEL	Static config or channel catalog.
GLOBAL	Only allowed for approved admin roles.

Execution:

1. Validate admin has `rbac.manage`.
2. Validate target user.
3. Validate scope value exists and is active.
4. Reject GLOBAL unless admin and target role policy allows it.
5. Insert `rbac_user_scope_assignment`.
6. Insert audit row `USER_SCOPE_ASSIGNED`.
7. Insert outbox event `RbacUserScopeAssigned`.
8. Invalidate user's effective-access cache.
9. Commit.

Response:

```
{
  "scopeAssignmentId": "usa_01J_ALICE_FR002",
  "authUserId": "kc_user_alice",
  "scopeType": "FRANCHISE",
  "scopeValue": "fr_01J_NRB_002",
  "active": true
}
```

6. Flow 4 - Frontend Loads Effective Access

6.1 User Opens Backoffice

1. Browser redirects to Keycloak.
2. User logs in.
3. Keycloak returns access token.
4. Backoffice calls:

```
GET /api/rbac/users/kc_user_alice/effective-access?appCode=FE-APP-01
Authorization: Bearer <jwt>
```

6.2 RBAC API Computes Access

1. Validate JWT.
2. Confirm requested `auth_user_id` matches JWT subject, unless caller has admin permission.
3. Load active user role assignments.
4. Load role permissions.
5. Apply explicit DENY entries after ALLOW.
6. Load active scope assignments.
7. Load frontend actions for FE-APP-01.
8. Mark actions visible/enabled if required permission exists and feature flag allows it.
9. Cache result for short TTL, for example 2-5 minutes.
10. Return access matrix.

Response:

```
{
  "authUserId": "kc_user_alice",
  "operatorCode": "WIK",
```

```

"roles": ["CUSTOMER_CARE_AGENT"],
"permissions": ["customer.read", "ticket.read", "ticket.create"],
"scopes": [
  {
    "scopeType": "FRANCHISE",
    "scopeValue": "fr_01J_NRB_002",
    "scopeLabel": "Nairobi 002"
  }
],
"frontendActions": [
  {
    "actionCode": "backoffice.customers.search",
    "visible": true,
    "enabled": true
  },
  {
    "actionCode": "backoffice.tickets.create",
    "visible": true,
    "enabled": true
  }
]
}

```

Frontend behavior:

- Show menus/actions from frontendActions.
- Do not treat hidden buttons as security.
- Include JWT on every API call.
- Handle 403 from backend even if the UI thought the action was allowed.

7. Flow 5 - Module Enforces Permission And Scope

Example: TCK create ticket.

7.1 User Sends API Request

```

POST /api/tickets
Authorization: Bearer <jwt>
Idempotency-Key: ui-ticket-01J
Content-Type: application/json

{
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0ZZ2N4P",
  "typeCode": "TECHNICAL_SUPPORT",
  "categoryCode": "NO_INTERNET",
  "sourceChannel": "CALL_CENTER",
  "subject": "No internet"
}

```

7.2 TCK API Validates

1. Validate JWT locally through FOUNDATION_AUTH middleware.
2. Extract authUserId, operatorCode, and coarse roles.
3. Resolve target franchise/scope:
 - by customer/account if ILM returns franchise context;
 - by HomePass franchise assignment if HomePass is present;
 - or by ticket queue policy.
4. Call RBAC check:

```

POST /api/rbac/check
Authorization: Bearer <service-token>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "authUserId": "kc_user_alice",
  "permissionCode": "ticket.create",
  "target": {

```

```

        "scopeType": "FRANCHISE",
        "scopeValue": "fr_01J_NRB_002"
    }
}

```

5. If RBAC returns DENY, return 403.
6. If RBAC returns ALLOW, continue TCK business validation.
7. Process ticket creation.

RBAC response:

```

{
  "decision": "ALLOW",
  "reason": "ROLE_PERMISSION_AND_SCOPE_MATCH",
  "matchedRoles": ["CUSTOMER_CARE_AGENT"],
  "matchedScopes": ["fr_01J_NRB_002"]
}

```

Important: the module owns business validation. RBAC only decides whether the user may attempt the action in the target scope.

8. Flow 6 - Revoke Role Or Scope

Revoke by ending the assignment, not deleting the row.

```

DELETE /api/rbac/user-role-assignments/ura_01J_ALICE_CC
Authorization: Bearer <jwt>
Idempotency-Key: revoke-alice-cc-agent
Content-Type: application/json

{
  "reasonCode": "TRANSFERRED_TO_BILLING",
  "effectiveTo": "2026-05-29T15:00:00Z"
}

```

Execution:

1. Validate caller has `rbac.manage`.
2. Lock assignment row.
3. Set `active = false, effective_to`.
4. Insert audit row `USER_ROLE_REVOKED`.
5. Insert outbox event `RbacUserRoleRevoked`.
6. Invalidate effective-access cache for user.
7. Commit.

The user's current JWT may still exist until token expiry. High-risk modules should call RBAC for fresh checks, so revocation takes effect without waiting for token expiry.

9. Keycloak Synchronization

If the operator wants Keycloak groups to drive baseline assignments:

1. Store mapping in `rbac_role.keycloak_role_name`.
2. Run scheduled sync every 5-15 minutes, or consume Keycloak admin events if configured.
3. For each user/group change:
 - create missing `rbac_user_role_assignment`;
 - end assignments no longer present if source is `KEYCLOAK_GROUP`;
 - do not overwrite assignments created manually with source `BACKOFFICE_ADMIN`.
4. Emit normal RBAC events and audit rows.

Do not query Keycloak on every module request. Runtime requests use JWT and RBAC DB/cache.

10. SQL Migration Sketch

Use the project migration tool. Conceptual schema:

```
CREATE TABLE rbac_role (
  role_id text PRIMARY KEY,
  operator_code text NOT NULL,
  role_code text NOT NULL,
  display_name text NOT NULL,
  role_family text NOT NULL,
  description text NOT NULL,
  status text NOT NULL,
  keycloak_role_name text,
  created_at timestampz NOT NULL,
  updated_at timestampz NOT NULL,
  UNIQUE (operator_code, role_code)
);

CREATE TABLE rbac_permission (
  permission_id text PRIMARY KEY,
  permission_code text NOT NULL UNIQUE,
  module_code text NOT NULL,
  action_group text NOT NULL,
  risk_level text NOT NULL,
  description text NOT NULL,
  scope_required boolean NOT NULL DEFAULT true,
  status text NOT NULL,
  created_at timestampz NOT NULL
);

CREATE TABLE rbac_role_permission (
  role_permission_id text PRIMARY KEY,
  role_id text NOT NULL REFERENCES rbac_role(role_id),
  permission_id text NOT NULL REFERENCES rbac_permission(permission_id),
  effect text NOT NULL,
  condition_json jsonb NOT NULL DEFAULT '{}':::jsonb,
  active boolean NOT NULL DEFAULT true,
  created_at timestampz NOT NULL,
  UNIQUE (role_id, permission_id)
);

CREATE TABLE rbac_user_role_assignment (
  assignment_id text PRIMARY KEY,
  operator_code text NOT NULL,
  auth_user_id text NOT NULL,
  role_id text NOT NULL REFERENCES rbac_role(role_id),
  source text NOT NULL,
  effective_from timestampz NOT NULL,
  effective_to timestampz,
  active boolean NOT NULL DEFAULT true,
  created_by_user_id text NOT NULL,
  created_at timestampz NOT NULL
);

CREATE TABLE rbac_user_scope_assignment (
  scope_assignment_id text PRIMARY KEY,
  operator_code text NOT NULL,
  auth_user_id text NOT NULL,
  scope_type text NOT NULL,
  scope_value text NOT NULL,
  scope_label text,
  effective_from timestampz NOT NULL,
  effective_to timestampz,
  active boolean NOT NULL DEFAULT true,
  created_by_user_id text NOT NULL,
  created_at timestampz NOT NULL
);
```

Add indexes:

```
CREATE INDEX idx_rbac_user_role_active
  ON rbac_user_role_assignment(operator_code, auth_user_id, active);

CREATE INDEX idx_rbac_user_scope_active
  ON rbac_user_scope_assignment(operator_code, auth_user_id, active);

CREATE INDEX idx_rbac_permission_code
  ON rbac_permission(permission_code);
```

11. Cache Strategy

Cache key:

```
rbac:effective:{operatorCode}:{authUserId}:{appCode}
```

Recommended TTL:

2-5 minutes

Invalidate cache when these events occur:

- RbacUserRoleAssigned
- RbacUserRoleRevoked
- RbacUserScopeAssigned
- RbacUserScopeRevoked
- RbacRolePermissionChanged

For high-risk actions, modules can bypass cache or require a recent access snapshot.

12. Error Handling

Case	Response
Missing JWT	401 UNAUTHORIZED.
Valid JWT but no rbac.manage for admin API	403 FORBIDDEN.
Duplicate role code	409 CONFLICT.
Unknown permission code	422 UNPROCESSABLE_ENTITY.
Assign inactive role	409 CONFLICT.
Assign unknown franchise scope	422 UNPROCESSABLE_ENTITY.
Same idempotency key with different body	409 CONFLICT.
RBAC check denied	200 with decision = DENY for check API; module converts to 403.
Effective access requested for another user without admin permission	403 FORBIDDEN.

13. Required Tests

Test	Expected result
Create role	Role row, audit row, outbox event created.
Create duplicate role	409.
Add permission to role	Role-permission row and event created.
Assign role to user	Assignment, audit, event, cache invalidation.
Assign scope to valid franchise	Scope row created.
Assign scope to unknown franchise	422.
Effective access with role + scope	Returns permissions and scope.
Check permission with matching scope	ALLOW.
Check permission without matching scope	DENY.
Revoke role	User loses permission on next effective access.
Revoke scope	Permission remains but target-scope check denies.
Frontend action matrix	Only actions whose permissions are granted are visible/enabled.
High-risk permission change	Audit row exists and event emitted.

14. Implementation Checklist

1. Create schema and seed catalog rows.

2. Implement role/permission CRUD.
3. Implement role-permission APIs.
4. Implement user role assignment APIs.
5. Implement user scope assignment APIs with reference validation.
6. Implement effective-access service.
7. Implement RBAC check and bulk-check APIs.
8. Implement frontend action matrix.
9. Implement outbox events and cache invalidation.
10. Add Keycloak group synchronization if required.
11. Add module middleware/helper library for permission checks.
12. Patch module DDs to reference permission codes where high-risk endpoints exist.
13. Patch frontend docs to read role/action matrix from EM-CFG-03.

15. What Not To Build In V1

- Do not build an approval workflow engine here. EM-CFG-04 owns approval catalog.
- Do not encode billing thresholds, write-off limits, or discount eligibility here.
- Do not query Keycloak on every request.
- Do not rely only on JWT coarse roles for high-risk business actions.
- Do not expose role administration to ordinary supervisors unless explicitly permitted.
- Do not let customer self-care inherit staff RBAC roles.